

NHÓM 10

Resource Design & Data Modeling

Nội dung chính

01

Thiết kế Resource Tree

Nguyên tắc thiết kế cấu trúc tài nguyên phân cấp và quản lý relationships

02

Chiến lược Phân trang

Offset/Limit, Cursor-based, Page-based pagination

03

So sánh và Lựa chọn

Phân tích ưu/nhược điểm để lựa chọn chiến lược phù hợp

Thiết kế Resource Tree

Nguyên tắc cơ bản của Resource Modeling



1. Xác định tài nguyên

Xác định các tài nguyên chính trong hệ thống. Ví dụ: `books`, `users`, `borrows`



2. Định nghĩa quan hệ

Thiết lập mối quan hệ giữa các tài nguyên: `users` có quan hệ nhiều-nhiều với `books` được thể hiện qua `borrows`.



3. Thiết kế URIs

Tạo URIs có ý nghĩa, nhất quán phản ánh phân cấp và quan hệ giữa các `resources`

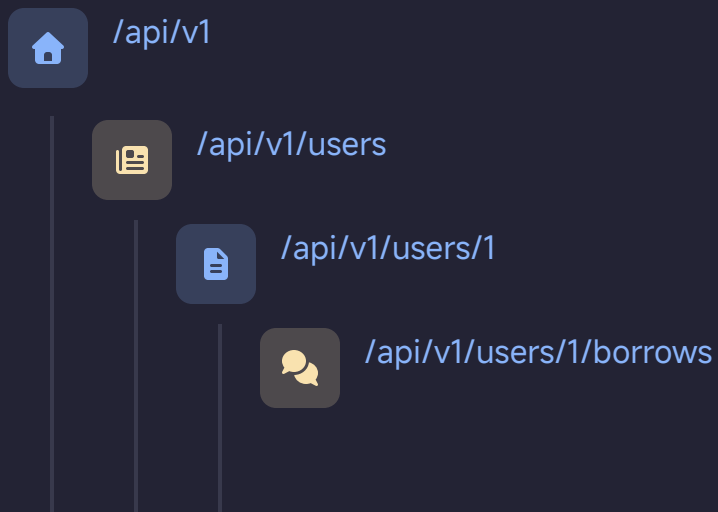


4. Biểu diễn tài nguyên

Chọn định dạng phù hợp (thường là JSON) để biểu diễn.

Resource Tree

Ví dụ: Book Management API



Lợi ích

- ✓ Dễ hiểu
- ✓ Dễ đoán
- ✓ Self-documenting API

GET `/api/v1/users/{user_id}/borrows` View list of borrowed books



POST `/api/v1/users/{user_id}/borrows` Borrow a book



Quản lý quan hệ giữa các resource



Resource Embedding

Nhúng resources trực tiếp vào parent resource response

```
GET /api/v1/users/1/borrows

"data": [
  {
    "id": 1,
    "title": "the art of war",
    "author": "sun tzu",
    "year": 450,
    "borrow_date": "2026-04-02T02:48:38.035032"
  }
]
```

- + Ưu điểm:** Giảm số requests, data đầy đủ trong một lần gọi
- Nhược điểm:** Response lớn, over-fetching



Resource Identifiers

Sử dụng id để thể hiện relationships

```
GET /api/v1/users/1/borrows

{
  "metadata": {
    "apiVersion": "1.0",
    "timestamp": "2026-04-02T02:48:42.940164"
  },
  "data": [{"id": 1}]
}
```

- + Ưu điểm:** Response ngắn, dễ cache từng tài nguyên
- Nhược điểm:** Nhiều requests hơn để lấy đầy đủ data

02

Chiến lược Phân trang

Các phương pháp phân trang dữ liệu trong REST API

Tại sao cần Phân trang?



Hiệu năng

Load toàn bộ dataset có thể làm quá tải servers và client applications, dẫn đến response chậm

Vấn đề:

- Query chậm
- Memory overhead lớn
- Network bandwidth cao



Dễ sử dụng

Users khó điều hướng và hiểu dataset lớn khi tất cả hiện thị cùng 1 lúc

Giải pháp:

- Chia nhỏ thành các phần
- Dễ dàng điều hướng (next/prev/jump)



An toàn

Giới hạn data trong mỗi API call giúp bảo vệ thông tin nhạy cảm

Lợi ích:

- Giảm khả năng bị tấn công
- Rate limiting hiệu quả hơn



Những trường hợp cần phân trang

- ✓ Social media feeds
- ✓ E-commerce catalogs
- ✓ Transaction lists
- ✓ Search results

Offset-Based Pagination

Cơ chế hoạt động

Sử dụng **offset** (vị trí bắt đầu) và **limit** (số lượng records) để xác định data cần lấy

Request:

```
GET /items?offset=10&limit=10
```

Lấy 10 items bắt đầu từ vị trí thứ 10

SQL Query:

```
SELECT * FROM items
ORDER BY id
LIMIT 10 OFFSET 10
```



Tính toán Offset

$$\text{offset} = (\text{page_number} - 1) \times \text{page_size}$$

Ví dụ: Page 3 với page size 10 → offset = 20

Response Format

```
{
  "data": [...],
  "meta": { "total": 1000, "limit": 10, "offset": 10 },
  "links": {
    "self": "/items?offset=10&limit=10",
    "next": "/items?offset=20&limit=10",
    "prev": "/items?offset=0&limit=10",
    "first": "/items?offset=0&limit=10",
    "last": "/items?offset=990&limit=10"
  }
}
```

Ưu điểm & Nhược điểm

+ Ưu điểm

- Đơn giản, dễ cài đặt
- Native support trong SQL
- Có thể jump đến bất kỳ page nào

- Nhược điểm

- Performance giảm khi offset lớn
- Data drift khi có thay đổi
- Truy vấn COUNT(*) có chi phí cao

Page-Based Pagination

Cơ chế hoạt động

Sử dụng **page** parameter để chỉ định số thứ tự trang muốn lấy

Request:

```
GET /items?page=2&limit=20
```

Lấy page thứ 2, mỗi page 20 items

Công thức tính offset:

$\text{offset} = (\text{page} - 1) \times \text{limit}$

Ví dụ: Page 2, limit 20 $\rightarrow$ offset = 20

Response với Page Metadata

```
{
  "data": [...],
  "pagination": {
    "current_page": 2,
    "per_page": 20,
    "total_pages": 50,
    "total_items": 1000
  },
  "links": {
    "first": "/items?page=1&limit=20",
    "prev": "/items?page=1&limit=20",
    "next": "/items?page=3&limit=20",
    "last": "/items?page=50&limit=20"
  }
}
```

UI/UX Friendly



Thao tác quen thuộc với users

So sánh với Offset-Based

== **Giống:** Cả hai đều dựa trên offset/limit SQL

≠ **Khác:** Page-based dễ hiểu hơn cho end users

⚠ **Lưu ý:** Vẫn có vấn đề hiệu năng và data drift như offset-based

Cài đặt Page-based Pagination

Tạo RequestParser

```
pagination_parser = reqparse.RequestParser()
pagination_parser.add_argument('page', type=int, default=1, help='Page number')
pagination_parser.add_argument('per_page', type=int, default=10, help='Item per page')
pagination_parser.add_argument('q', type=str, help='Search query')
```

Lấy các args từ URL

```
@ns_books.route('')
class BookList(Resource):
    @jwt_required()
    @ns_books.expect(pagination_parser)
    def get(self):
        """Get all books"""
        args = pagination_parser.parse_args()
        page = args['page']
        per_page = args['per_page']
        query = args.get('q')
```

Lấy dữ liệu

```
filtered_books = [b for b in books_db if query in b['title'].lower()] if query else books_db
total_elements = len(filtered_books)

# page = ceil(ele / per)
total_pages = (total_elements + per_page - 1) // per_page if total_elements > 0 else 0

if page > total_pages and total_pages > 0:
    page = total_pages

start = (page - 1) * per_page
end = min(start + per_page, total_elements)
items = filtered_books[start:end]
```

Tính toán prev, next và trả về dữ liệu

```
pagination = {
    "type": "page-based",
    "current_page": page,
    "per_page": per_page,
    "total_elements": total_elements,
    "total_pages": total_pages,
    "links": {
        "next": f"?page={page+1}&per_page={per_page}" + (f"&q={query}" if query != None else "") if page < total_pages else None,
        "prev": f"?page={page-1}&per_page={per_page}" + (f"&q={query}" if query != None else "") if page > 1 else None
    }
}
return wrap_with_metadata(items, pagination), 200
```

Cài đặt Page-based Pagination

Kết quả

api/v1/books Book Management

GET /api/v1/books Get all books

Parameters

Name	Description
page integer (query)	Page number 2
per_page integer (query)	Item per page 1
q string (query)	Search query a

Execute Clear

```
"metadata": {
  "apiVersion": "1.0",
  "timestamp": "2026-04-02T03:41:26.342307",
  "pagination": {
    "type": "page-based",
    "current_page": 2,
    "per_page": 1,
    "total_elements": 3,
    "total_pages": 3,
    "links": {
      "next": "?page=3&per_page=1&q=a",
      "prev": "?page=1&per_page=1&q=a"
    }
  }
},
"data": [
  {
    "id": 2,
    "title": "the c programming language",
    "author": "brian kernighan",
    "year": 1978,
    "borrow_date": "2026-04-02T02:46:11.088598"
  }
]
```

Cursor-Based Pagination

Cơ chế hoạt động

Sử dụng **cursor** (unique identifier/token) để đánh dấu vị trí trong dataset

Request:

```
GET /items?cursor=abc123&limit=10
```

Cursor là ID của last item từ page trước

SQL Query:

```
SELECT * FROM items
WHERE id > 'abc123'
ORDER BY id ASC
LIMIT 10
```

</> Cursor Encoding

Cursor thường được encode sang base64:

user:W07QCRPA4 ➡ dXNlcjpxMDdRQ1JQQTQ=

Response Format

```
{
  "data": [...],
  "next_cursor": "xyz789",
  "limit": 10,
  "links": {
    "self": "/items?cursor=abc123&limit=10",
    "next": "/items?cursor=xyz789&limit=10",
    "prev": "/items?cursor=aaa112&limit=10",
  }
}
```

Ưu điểm & Nhược điểm

+ Ưu điểm

- Độ phức tạp $O(1)$ – ổn định ở mọi page
- Không bị data drift
- Không cần truy vấn COUNT
- Hoàn hảo cho infinite scroll

- Nhược điểm

- Không thể nhảy đến page bất kỳ
- Khó hơn để cài đặt

03

So sánh và Lựa chọn

Phân tích ưu/nhược điểm và hướng dẫn lựa chọn chiến lược phù hợp



So sánh hiệu năng: Offset vs Cursor



Offset-Based: $O(N)$

Database phải **scan và xóa** tất cả rows từ đầu đến offset

Ví dụ: OFFSET 10000

1. Scan 10,000 rows
2. Discard 10,000 rows
3. Return 10 rows

- ❌ Page 1: -10ms
- ❌ Page 1000: -3000ms
- ⚠️ Chậm dần theo offset



COUNT(*) Query

Offset-based thường cần COUNT(*) để tính total pages → **tốn kém trên bảng lớn**



Cursor-Based: $O(1)$

Database dùng **index** để jump trực tiếp đến cursor record

Ví dụ: cursor=xyz789

1. Index lookup: $O(1)$
2. Jump to cursor position
3. Return next 10 rows

- ✅ Page 1: -10ms
- ✅ Page 10000: -10ms
- ✅ Performance ổn định

Vấn đề Data Drift



Offset-Based: Data Drift

Khi data thay đổi (thêm/xóa) giữa các requests, người dùng có thể thấy **trùng lặp** hoặc **thiếu records**

Ví dụ:

Request 1: GET /items?offset=0&limit=5
→ Items: [A, B, C, D, E]

⚠ Thêm X vào trước A

Request 2: GET /items?offset=5&limit=5
→ Items: [E, F, G, H, I]

✗ E bị trùng!



Cursor-Based: Consistent

Cursor trở đến **record nhất định**, không phải vị trí

Ví dụ:

Request 1: GET /items?limit=5
→ Items: [A, B, C, D, E], cursor=E

⚠ Thêm X vào trước A

Request 2: GET /items?cursor=E&limit=5
→ Items: [F, G, H, I, J]

✅ Không trùng, không thiếu!



Khi nào quan trọng?

- Giao dịch tài chính
- Real-time activity feeds trên các mxh
- Lịch sử chat
- Audit logs

Khi nào dùng Offset-Based Pagination?



1. Dataset nhỏ

Dưới **10,000 dòng** – hiệu năng vẫn tốt, không cần tối ưu phức tạp



2. UI cần "Jump to Page"

Admin dashboards – users cần nhảy đến page cụ thể



3. Dữ liệu tĩnh, ít thay đổi

Archived logs, reference data – **data drift** không phải vấn đề



4. Prototypes & Internal Tools

Khi cần nhanh chóng phát triển phần mềm – **nhANH, đơn giản**



5. Search Results

Users **hiếm khi đi quá page đầu** – offset nhỏ nên hiệu năng vẫn tốt



Quy tắc

Offset ổn cho **dataset nhỏ, tĩnh**. Khi dataset lớn hoặc thay đổi liên tục → chuyển sang Cursor.

Khi nào dùng Cursor-Based Pagination?



1. Dataset lớn

Hàng triệu dòng – cursor giữ hiệu năng ổn định ở mọi page



2. Infinite Scroll / Load More

UI pattern phổ biến trên **mobile apps**, **social feeds** – không cần jump to page



3. Data thay đổi liên tục

Real-time feeds, **activity streams** – không cho phép trùng/thiếu

Ví dụ thực tế



Twitter API

Timeline feeds, tweets



Instagram Graph API

Posts, comments, likes



Slack API

Messages, channels



Facebook API

News feed, notifications

Query Parameters bổ sung



Filtering

Lọc data trước khi phân trang

```
?status=active
?category=technology
?date_from=2024-01-01
?date_to=2024-12-31
```

Ví dụ:
`/posts?category=tech&page=1`



Sorting

Sắp xếp data trước khi phân trang

```
?sort=created_at
?sort=-created_at
?sort=name&order=asc
?sort=-priority,created_at
```

Ví dụ:
`/posts?sort=-created_at&limit=10`



Search

Tìm kiếm text trong data

```
?q=keyword
?search=rest+api
?query=pagination
```

Ví dụ:
`/posts?q=rest+api&page=1`

Kết hợp tất cả parameters

```
GET /api/posts?category=technology&status=published&sort=-created_at&q=rest+api&cursor=abc123&limit=20
```



Filter: category, status



Sort: -created_at



Search: rest api



Paginate: cursor, limit